OP_NpcMoveUpdate

Purpose

Per-NPC movement update. Delta-style, frequent. Sent by the zone server to keep clients informed of mob positions between authoritative snapshots. The most common packet in any capture.

Identification evidence

Frequency. Highest of any opcode by a wide margin. In any capture with active mobs, this is the top row in the opcode list by count.

Size. Historically the base payload has been around 19 bytes with optional fields extending it. The size distribution shows a sharp mode at the base size with a tail. Subject to per-patch verification.

Direction. Z2C. Never sent by the client.

Context. Fires continuously while any NPC is moving. Frequency drops to near zero in empty areas.

Disambiguation from OP_MobUpdate. NpcMoveUpdate has the larger base size and the optional-field tail. MobUpdate is fixed-shape and smaller.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify (presence, position, encoding, divisor):

- Spawn ID near the start of the payload
- Position fields (x, y, z) — encoding and per-axis divisors
- Heading — bit width, encoding, divisor
- Pitch, velocity, heading delta, dx/dy/dz — in the optional group
- Optional group bit offset
- Optional bit-to-field mapping (May 2026 patch scrambled this without touching base layout)

Verification technique. Capture a session with mobs visible. Decode consecutive NpcMoveUpdate packets for the same spawn ID. Positions should advance continuously, not scatter. If positions look sensible, base layout is intact. If positions scatter, a coordinate field has shifted.

Mobs with known walking patterns help — North Karana druid roamers are one observed example. No table of predictable-path mobs exists yet.

Operator workflow

1. Open the opcode list. Sort by count descending. Top row is the candidate.
2. Open the analysis tab. Confirm size distribution has a sharp mode with a tail.
3. Open the trace tab. Verify the candidate fires continuously while mobs are visible.
4. Pick a spawn observable in-game. Capture a sequence of NpcMoveUpdate packets for that spawn ID.
5. Decode the sequence. Compare to observed motion.
6. If positions are wrong, two-packet diff between a prior-patch sample and a current sample.
7. Once base layout is verified, work through optional sub-fields one bit at a time.

Known fragilities

- **Spawn ID byte order** differs from OP_MobUpdate and OP_ZoneEntry. NpcMoveUpdate reports byte-swapped IDs. Stable for several patches but worth re-confirming.
- **Optional fields scrambled in May 2026 patch.** First documented case of optional-bit-to-field mapping changing without base layout changing.

Dependencies

None for identification. For verifying field contents (spawn ID cross-check), having identified OP_ZoneEntry first helps but is not required.

OP_MobUpdate

Purpose

Periodic mob position snapshot. Sent by the zone server as an authoritative correction to dead-reckoning positions clients maintain from NpcMoveUpdate. Lower frequency than NpcMoveUpdate, fixed-shape payload.

Identification evidence

Frequency. Second-highest mob-related opcode after NpcMoveUpdate. Steady cadence rather than event-driven.

Size. Historically around 14 bytes, fixed. Tight distribution. Subject to per-patch verification.

Direction. Z2C

Context. Continues firing when mobs are stationary. The complement of NpcMoveUpdate's burstiness.

Disambiguation from OP_NpcMoveUpdate. MobUpdate is fixed-length and steady. NpcMoveUpdate is variable-length and event-driven. If two high-frequency opcodes are candidates, the one with the tighter size distribution is MobUpdate.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Spawn ID
- Coordinate fields (x, y, z) — historically bit-packed (19 bits each, 3 fractional bits) but not assumed across patches
- Any additional fields beyond coordinates

Verification technique. MobUpdate's authoritative-snapshot role makes it cross-checkable against NpcMoveUpdate. For a given spawn ID, MobUpdate positions should land near where NpcMoveUpdate dead-reckoning predicted, within tolerance. If one opcode is decoded correctly and the other isn't, the decoded one serves as ground truth for the other.

Operator workflow

1. Open the opcode list. The next highest mob-frequency candidate after NpcMoveUpdate with a tight size distribution is MobUpdate.
2. Open the analysis tab. Confirm size distribution is essentially a single value.
3. Open the trace tab. Verify steady cadence while mobs are present — should continue even when no mobs are moving.
4. Pick a spawn observable in-game. Decode its MobUpdate entries. Compare to observed position.
5. If NpcMoveUpdate is already identified, cross-check positions between the two opcodes for the same spawn.
6. If positions are wrong, two-packet diff between a prior-patch sample and a current sample.

Known fragilities

- **Bit-packed coordinates.** Historically 19 bits per axis with byte-spanning. One-bit shift early in the payload misaligns everything downstream.
- **Spawn ID byte order.** Agrees with OP_ZoneEntry. Differs from OP_NpcMoveUpdate. Stable but verify per patch.

Dependencies

None for identification. For content verification, having OP_NpcMoveUpdate decoded provides cross-check via shared spawn IDs and agreeing positions.

OP_PlayerProfile

Purpose

Bulk character profile load. Sent by the zone server once per character at zone-in. Contains authoritative state of the player character. Variable-length, distinctive size signature.

Identification evidence

Size is a strong signal. Historically around 23 KB, variable. After a zone-in, the first packet in the 20 KB range arriving on a session is a good PlayerProfile candidate.

Frequency. Very low — one per character per zone-in. In a six-character capture covering one zone-in, exactly six PlayerProfile packets should appear.

Direction. Z2C

Context. Fires during the post-session-establishment burst after a zone-in.

Name presence as confirmation. Character names are known in advance to the operator but not yet associated with session port pairs when the profile arrives. Searching a candidate packet for any of the known character names confirms PlayerProfile and binds that session to that character. The name appears in the middle of the payload.

A 20 KB-range variable-length packet containing one of the known character names is essentially a confirmed PlayerProfile.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields with database anchors available for cross-check:

- Character name (ASCII, null-terminated)
- Class byte (note EQClass enum is off-by-one from wire values)
- Level
- HP, mana
- Position (x, y, z, heading)
- Practice points, ability scores
- Coin block

Verification technique. Glass has stored last-known character state in the database. For each character, that stored state provides multiple independent anchor values to search for in their PlayerProfile packet. With six characters and several anchor fields per character, agreement on an offset across all six characters confirms a field at that offset.

Profile contents — known and unknown

The profile contains the verified fields listed above plus other contents not yet identified. No arrays are known to be present. Variable length is real but the source has not been pinned down.

Inventory has historically been believed to live elsewhere, not in PlayerProfile.

For unidentified profile contents, the diff-after-change technique applies: capture a profile with a known state, change something controllable, zone again to force a fresh profile, capture again, diff the two.

Operator workflow

1. Ensure Glass database has current character state for the characters in the capture.
2. Capture a session with multiple characters zoning into the same zone simultaneously.
3. After zone-in, look for the first packet on each session in the 20 KB range.
4. For each candidate, search for any known character name. A match confirms PlayerProfile and identifies which character that session belongs to.
5. With PlayerProfile identified per session, use database anchors to locate other known fields.
6. For unknown profile contents, use the diff-after-change technique with two captures bracketing a controlled state change.

Known fragilities

- **Field offsets drift across patches** but tend not to scramble wholesale. Most fields stay near their prior offset.
- **Variable-length sections** make absolute offsets unreliable past those sections. Fields after a variable section need re-anchoring per patch.

Dependencies

Identification: None. Size + name search is self-contained.

Verification: Requires up-to-date character state in the Glass database. Database currency is a precondition for patch-day PlayerProfile verification.

OP_CommonMessage

Purpose

Player-originated chat messages. Carries say, shout, OOC, /tell, and similar channels. Both inbound and outbound. Distinguished from server-formatted messages.

Identification evidence

Triggerability. Operator can produce packets on demand. Send a /say from any character; one CommonMessage appears.

Size. Variable, dominated by message length. Size scales linearly with message content.

Frequency. Sporadic, fully under operator control during patch-day investigation.

Direction. Z2C, C2Z both.

Channel disambiguation. Multiple channel types share the opcode. Historically the channel is encoded as a single byte. Send identical text on different channels back-to-back and diff the resulting packets; the byte that changes is the channel discriminator.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields with operator-controlled anchors:

- Sender name (ASCII, null-terminated)
- Message text (ASCII)
- Channel discriminator (byte)
- Target name for private messages

Verification technique. Send a sequence of messages with operator-chosen text. The sent text appears literally in the packet. Searching for the known text locates the message field. Searching for the known sender name locates the sender field. The channel byte is found by sending identical text on different channels and diffing.

The most cleanly anchored opcode in the set — every interesting field has a value the operator just typed.

Operator workflow

1. Open the trace tab.
2. Send a /say with distinctive text. Note the timestamp.
3. The packet at that timestamp on the operator's session is the CommonMessage candidate (outbound direction).
4. Confirm by searching for the exact text typed.
5. Locate the sender name field by searching for the known character name.
6. Locate the channel byte by sending identical text on /say vs /ooc vs /shout and diffing.
7. Repeat for inbound direction.

Known fragilities

- **Channel byte position** has moved between patches.
- **String encoding** has historically been ASCII null-terminated. Verify each patch.
- **Tell vs say payload shape.** Tells carry a target name. Layout differs by channel.

Dependencies

None. CommonMessage can be the first opcode identified post-patch.

OP_FormattedMessage

Purpose

Server-originated formatted messages. Carries template IDs plus argument strings; the client substitutes them into localized format strings. Used for spell-casting feedback, system messages, faction changes, and tracking output. Not combat-driven.

Identification evidence

Triggerability. Operator-controllable. Cast a spell, gain a faction, activate tracking — each produces FormattedMessage packets.

Size. Variable, but smaller than CommonMessage on average. Payload carries template IDs and arguments rather than fully rendered text.

Frequency. Sporadic. Triggered by specific events.

Direction. Z2C

Context. Triggered by specific events (spell cast, tracking spell active, faction shift, system events). Not a passive combat stream.

Disambiguation from CommonMessage. CommonMessage carries literal text; FormattedMessage carries template IDs and arguments. Visually different on the wire.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Template/message ID (numeric)
- Argument count or sentinel-terminated argument list
- Argument strings (ASCII, length-prefixed or null-terminated)
- Channel/color discriminator

Verification technique. Trigger a specific known message. The argument string appears in the packet. Repeat the same trigger; the template ID stays constant, argument values vary across instances.

Tracking is the strongest trigger for FormattedMessage identification: ranger or druid Track activation produces multiple FormattedMessage packets with diverse argument content (different mob names per entry). The repeated structure across many packets with varying string content reveals argument layout cleanly. Spell-cast feedback also produces FormattedMessages but in smaller bursts.

Operator workflow

1. Open the trace tab.
2. On a ranger or druid, activate Track. Multiple FormattedMessage packets should arrive carrying mob names.
3. Search the packets for any name known to be in the zone. Match confirms the opcode.
4. With multiple captured packets, identify which bytes are constant (template ID, formatting structure) and which vary (argument values).
5. For numeric arguments, the value matches what the client UI displayed.

Known fragilities

- **Template ID table** is itself patch-mutable. Anchor on argument content rather than template ID.
- **Argument encoding** (length-prefixed vs null-terminated, fixed-width vs variable) is patch-fragile.

Dependencies

None for identification. Tracking-based identification benefits from a ranger or druid character.

OP_Death

Purpose

Notification that a spawn has died. Sent by the zone server when any spawn in the zone dies — NPC or player. Carries the identity of who died and who killed them.

Identification evidence

Triggerability. Operator-controllable. Kill a mob; one Death packet appears.

Size. Historically small and fixed. Two spawn IDs plus a small amount of additional data.

Frequency. Sporadic, operator-controllable during investigation.

Direction. Z2C

Context. Fires at the moment of a kill. No accompanying messages unless the operator sends one. Death is a discrete event on the wire.

Layout verification evidence

Field layouts are not assumed stable across patches. Death has been historically more stable than position-update opcodes, but sample size is small. Verify each patch.

Fields to verify:

- Victim spawn ID
- Killer spawn ID
- Any additional fields beyond the two IDs

Verification technique. Operator has multiple controllable anchors:

- The killer's spawn ID is the operator's own character's spawn ID (known from PlayerProfile or self-ZoneEntry).
- The victim's spawn ID can be cross-referenced from MobUpdate or NpcMoveUpdate packets for that mob immediately prior to its death.

Kill the same mob type repeatedly. Each Death packet should show killer ID constant (operator's character) and victim ID varying. The constant-vs-varying pattern identifies which field is which.

Layout comparison against stored field definitions. No stored prior-patch packet samples exist to diff against. The prior patch's field offsets and encodings are stored in the Inference DB. Verification is by decoded plausibility: decode a fresh Death packet using prior-patch field definitions, check whether decoded values are sensible (spawn IDs in expected range, matching observed kills).

Operator workflow

1. Identify a zone with controllable kills.
2. Open the trace tab.
3. Kill a mob. Note the timestamp.
4. Identify the candidate by size — small fixed-length packets in the post-kill window.
5. Apply prior-patch field definitions to the candidate. Check decoded values for plausibility.
6. Repeat with multiple kills. The killer ID should remain constant; the victim ID should vary.
7. If decoded values are nonsensical, the layout has shifted. Investigate by examining raw bytes — look for the operator's known spawn ID elsewhere in the payload.

Known fragilities

- **Spawn ID byte order.** Death historically agrees with ZoneEntry / MobUpdate byte order. Verify per patch.
- **Spawn ID width.** 16-bit historically.
- **Trailing fields beyond the two IDs.** Less firmly established. Damage value, skill used, environmental cause — investigation territory.

Dependencies

Identification: None.

Verification: Knowing the operator's own spawn ID and recent victim spawn IDs makes verification more direct. Without those, identification of which ID is which still works by constant-vs-varying across repeated kills.

OP_ZoneEntry

Purpose

Announcement that a spawn exists in the zone. Sent by the zone server once per spawn — during the post-zone-in burst (announcing every existing spawn) and on-demand later (announcing new spawns as they appear).

Identification evidence

Burst signature. The most distinctive context clue. After zone-in, the server sends one ZoneEntry per spawn currently in the zone — typically hundreds in a populated zone. Many packets of the same opcode arriving back-to-back, with similar structure varying in details.

Frequency. Heavy burst at zone-in, sporadic thereafter. The zone-in burst alone makes ZoneEntry the highest-count opcode in the first second after a zone-in.

Size. Variable — name strings cause length variation. Historically small-to-medium.

Direction. Z2C

Context. Bursts during zone-in, after PlayerProfile delivery. Sequence: session establishment → PlayerProfile → ZoneEntry burst → settled-state position updates.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Spawn ID
- Spawn name (ASCII)
- Level
- Class
- Race
- Position (x, y, z, heading) — historical presence uncertain
- Faction-related fields
- Flags (PC vs NPC, gender, anonymity, etc.)

Verification technique. Self-ZoneEntry is the strongest anchor. When the operator's character zones in, the server sends a ZoneEntry for that character. Searching the burst for the known character name locates the self-ZoneEntry. That packet's fields can be anchored against every value Glass has stored for that character.

Multi-anchor packet from a single capture, comparable to PlayerProfile's verification strength.

For other spawns in the burst: known NPC names (guards, merchants, named mobs) provide additional anchors. Level distribution across the burst is also a sanity check — most spawns should be in the zone's expected level range.

Operator workflow

1. Capture a session covering a zone-in.
2. Identify the candidate by burst signature: largest count in the first second after PlayerProfile delivery, variable but moderate payload size.
3. Within the burst, search for the operator's known character name. The matching packet is self-ZoneEntry.
4. Anchor every known field (name, level, class, race, etc.) against the character's database values.
5. Cross-check by searching other ZoneEntry packets for other known names (zone guards, named NPCs).
6. For unknown fields, the burst provides a large sample for distributional analysis — bytes constant across the burst are structural; bytes that vary but cluster around a few values are enumerations; bytes that vary widely are IDs or coordinates; bytes mostly zero with occasional non-zero are flags.

Known fragilities

- **Variable-length name field** makes absolute offsets unreliable past the name.
- **Position presence.** Historically uncertain whether ZoneEntry carries position.
- **Spawn ID byte order.** Agrees with OP_MobUpdate. Differs from OP_NpcMoveUpdate.
- **Flag bits and class/race byte assignments** drift between patches.

Dependencies

Identification: None.

Verification: Benefits from current Glass database state for the operator's characters.

OP_Target

Purpose

Notification of a target change. Carries the spawn ID of the new target. Single-field payload.

Identification evidence

Identification in general is impossible. A packet with a single small field doesn't carry enough internal structure to be distinguished by content from many other small opcodes. Identification depends entirely on operator action and timing correlation.

Triggerability is the only viable signal. Click on a known mob; one small packet appears at that timestamp containing that mob's spawn ID. The combination of:

- precise timestamp correlation with operator action,
- very small payload size,
- payload bytes matching a known target spawn ID,

is what identifies Target.

Size. Historically small and fixed.

Frequency. Sporadic, operator-controlled.

Direction. Z2C

Layout verification evidence

Single-field payload. Verification is confirming the bytes match the known spawn ID at the expected offset.

Verification technique. Target a sequence of mobs whose spawn IDs are known from recent ZoneEntry or MobUpdate traffic. Each Target packet should contain the corresponding ID. Vary the targeted mob and confirm the bytes change.

For the no-target sentinel: press escape, observe the resulting packet, note the sentinel value.

Operator workflow

1. Identify spawn IDs of nearby mobs from recent ZoneEntry or MobUpdate.
2. Open the trace tab.
3. Target a specific mob with a known spawn ID. Note the timestamp.
4. The small packet at that timestamp containing the known ID is Target.
5. Repeat with different targets to confirm field position is consistent.
6. Clear target to identify the no-target sentinel value.

Known fragilities

- **Spawn ID byte order.** Verify per patch.
- **Spawn ID width.** 16-bit historically.
- **No-target sentinel value.** Verify with an escape-key clear.

Dependencies

Requires known target spawn IDs from recent ZoneEntry or MobUpdate, plus operator action. Identification cannot succeed from packet content alone.

OP_HpUpdate

Purpose

Notification of an HP change for the operator's own character. Sent by the zone server when the character's HP changes. Carries the new HP value.

Identification evidence

Triggerability is value-based, not event-based. The server decides when to send updates. The operator influences the value, not the timing.

Size. Historically small.

Frequency. Variable. Quiet at full HP outside combat. Frequent when taking damage or healing.

Direction. Z2C

Context. Fires whenever the server wants to update the client on HP. No specific event context.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- HP value (numeric width and signedness must be verified)
- Character/spawn ID, if present
- Any additional fields

Verification technique: changes from a known baseline. The operator notes baseline HP. They cause an HP change with a known magnitude (take a hit from a specific mob, drink a known heal potion). The new HP value is baseline ± known delta. Search small inbound packets for a numeric field whose value differs from the prior packet of the same opcode by approximately the expected delta.

"Field changed by ~N" search is independent of width and encoding assumptions.

Multi-character corroboration. Multiple characters with known HP baselines — each one's HpUpdate carries that character's HP. Cross-character agreement on field offset confirms identification.

Operator workflow

1. Note baseline HP for the test character.
2. Open the trace tab.
3. Cause a known HP change. Note the expected new HP.
4. Identify small inbound packets on the operator's session in the window following the change.
5. Search for a numeric field whose value matches the expected new HP, or whose value differs from a prior packet by the expected delta.
6. Repeat with a second known change to confirm field position is consistent.

Known fragilities

- **HP value width and signedness.** Must be verified each patch.
- **Absolute vs delta encoding.** Search by absolute first.
- **Identifying the character.** Spawn ID may or may not be in the payload.

Dependencies

Requires baseline HP known to the operator and a known-magnitude HP change.

OP_ManaUpdate

Purpose

Notification of a mana change for the operator's own character. Sent by the zone server when the character's mana changes. Carries the new mana value.

Identification evidence

Triggerability is value-based, not event-based. Same as HpUpdate.

Size. Historically small.

Frequency. Variable. Depends on character activity. Casting triggers a change.

Direction. Z2C

Context. Fires whenever the server wants to update the client on mana. No specific event context.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Mana value (numeric width and signedness must be verified)
- Character/spawn ID, if present
- Any additional fields

Verification technique: changes from a known baseline. Same as HpUpdate. Note baseline mana, cast a spell of known cost, search for a numeric field that decreased by approximately that cost.

Operator workflow

1. Note baseline mana for the test character.
2. Open the trace tab.
3. Cast a spell with known mana cost. Expected new mana is baseline minus cost.
4. Identify small inbound packets on the operator's session in the window following the cast.
5. Search for a numeric field matching the expected new mana or differing from a prior packet by the expected delta.
6. Repeat with a second cast to confirm field position is consistent.

Known fragilities

- **Mana value width and signedness.** Must be verified each patch.
- **Absolute vs delta encoding.** Search by absolute first.
- **Identifying the character.** Spawn ID may or may not be in the payload.

Disambiguation from OP_HpUpdate

HpUpdate and ManaUpdate are structurally similar — both small inbound packets carrying a single numeric value. Disambiguation:

- Cause an HP change without mana change (take damage without casting). The opcode that fires is HpUpdate.
- Cause a mana change without HP change (cast a spell without taking damage). The opcode that fires is ManaUpdate.
- Once values are decoded, HP and mana have different baseline ranges for any given character.

Dependencies

Requires baseline mana known to the operator and a known-magnitude mana change.

OP_Tracking

Purpose

There are three versions (V0, V1, V2). V0 is a 0-length request sent C2Z. V1 is a fixed length, and V2 is variable length.

Response to a ranger or druid tracking skill activation. Sent by the zone server when the operator's character uses Track. Carries an array of trackable spawns currently in the zone.

Identification evidence

Content: V0 has no context. V1 has a "magic number" at a known offset. Otherwise V2. V2 is the only one of interest, and it has an array of structures prefixed by a count. Each entry has a spawn_id, a level, and a null-terminated string with a mob name. Strings should never be null.

Size. V0 is 0 bytes. V1 is fixed length (currently 80 bytes). V2 is variable length (header + array of variable length records).

Frequency. Sporadic, operator-controlled.

Direction. V0 is C2Z. V1 and V2 are Z2C.

Context. V1 and V2 follow a V0 from the client.

Layout verification evidence

Field layouts are not assumed stable across patches.

Element fields to verify:

- Spawn name (ASCII, length-prefixed or null-terminated)
- Spawn ID
- Level

Verification technique. Search the candidate packet for any known mob name — a named NPC known to live in this zone, a name from recent ZoneEntry traffic, or a name from local fauna. One ASCII match in a packet of the right size, following a tracking activation, confirms the opcode.

The operator does not need to exhaustively map every UI entry to a packet offset.

Once identified, element template verification proceeds by finding two or three known names in the same packet and computing offset spacing. Consistent spacing indicates fixed-width elements; varying spacing indicates variable-width.

Operator workflow

1. Move a ranger or druid character into a zone with trackable spawns.
2. Open the trace tab.
3. Activate Track. Note the timestamp.
4. The large inbound variable-length packet following activation is the Tracking candidate.
5. Search the packet for any name known to be in the zone. Match confirms the opcode.
6. For element template work, find a second known name and compute offset spacing.
7. With element boundaries identified, examine per-element bytes. Cross-reference candidate spawn IDs against recent ZoneEntry data.

Known fragilities

- **Element template structure** is patch-fragile independently of top-level array structure.
- **Name encoding.** Length-prefixed vs null-terminated affects element walking.
- **Pagination.** Whether large lists span multiple packets needs verification.
- **Spawn ID byte order within elements.** Verify per patch.

Dependencies

Identification: Requires a ranger or druid character and at least one name known to be in the zone.

Verification: Benefits from OP_ZoneEntry (spawn ID cross-reference) and OP_FormattedMessage (name corroboration through tracking-output messages).

OP_ClientUpdate

Purpose

Outbound position update from the client to the zone server. The operator's own character reports its position, heading, and movement state. The client-side counterpart to OP_NpcMoveUpdate. Sent on a regular cadence while in the zone.

Identification evidence

Direction is the primary discriminator. C2Z

Frequency. Steady cadence while in the zone. Even standing still, the client likely sends periodic ClientUpdate packets. This is a fairly high frequency packet.

Size. Fixed length. Currently at 42 bytes.

Context. Outbound traffic concurrent with the operator moving in-game.

Layout verification evidence

Field layouts are not assumed stable across patches. **The heading field shifted position in the May 2026 patch and has not yet been re-identified.**

Fields to verify:

- Position (x, y, z)
- Heading
- Pitch (if present)
- Velocity or movement state
- Spawn ID — operator's own character ID, possibly present

Verification technique. The operator's position is observable in-game via /loc and /heading commands.

- *Static position anchor.* Stand at a known position. Capture several ClientUpdate packets. Search for numeric values matching x, y, z. Fields should be at consistent offsets. Stand at a different position; same offsets should now hold the new values.
- *Movement diff*. Walk in a known direction (due north along a straight path). One coordinate should change monotonically; another should remain roughly constant.

Heading re-identification (priority for current patch). Face the character due north — heading value should be 0 or a known constant in EQ's convention. Rotate 90° to face east. Continue in 90° increments. Four known heading values give strong constraints on which bytes are the heading field.

Operator workflow

1. Move the operator's character to a known landmark with /loc readout.
2. Read off x, y, z and heading values.
3. Open the trace tab. Filter to outbound traffic on the operator's session.
4. Capture several ClientUpdate candidates while standing still.
5. Search for numeric values matching the known coordinates. Consistent offsets across packets indicate the coordinate fields.
6. For heading: face each cardinal direction, capture. Bytes that change in proportion to facing direction are the heading field.
7. For movement state and other fields: walk, jump, mount, sit, swim. Diff packets bracketing each state change.

Known fragilities

- **Heading field position shifted in the May 2026 patch.** Active investigation target.
- **Coordinate encoding.** Sign-magnitude vs two's-complement, bit width, divisor — all patch-fragile.
- **Cadence.** Whether the client always sends at the same rate, or backs off when stationary, may affect identification by frequency.

Dependencies

Identification: None. Direction (outbound) and frequency are sufficient.

Verification: Requires operator-known position values (/loc, /heading).

OP_SetChatServer

Purpose

Carries the network endpoint (host and port) of the chat server (UCS — Universal Chat Server) that the client should connect to for cross-zone chat channels and tell delivery. Sent by the zone or login server during session setup.

Identification evidence

Distinctive content. The payload contains an IP address or hostname and port number for the chat server. This is one of the few opcodes whose content is essentially unique — searching captured packets for a known chat server address narrows candidates immediately.

Frequency. Very low. Sent during session setup or when chat server configuration changes.

Size. Small. Bounded by hostname length plus a few additional fields.

Direction. Z2C

Context. Fires during the login or zone-in burst, around the same time as other configuration opcodes.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Chat server hostname or IP address (ASCII string)
- Port number (numeric)
- Possibly authentication tokens or server identifiers

Verification technique. The chat server address is observable from network captures independently — any chat-related traffic from the client will reveal where the client is sending it. That observed address can be searched for in candidate packets. A small inbound packet containing the chat server's hostname or IP in ASCII or stringified numeric form is the SetChatServer candidate.

Operator workflow

1. Capture a session covering login or zone-in.
2. Capture a small amount of subsequent chat-related traffic to observe the chat server's network address.
3. Search early inbound packets for the observed chat server hostname or IP.
4. The matching packet is the SetChatServer candidate.
5. With the candidate identified, examine surrounding bytes for the port number (small integer near the hostname).

Known fragilities

- **Address encoding.** Hostname (ASCII string) vs IPv4 dotted-quad vs raw 4-byte integer — all are possible representations.
- **Port encoding.** 16-bit integer, byte order to verify.
- **Authentication token presence.** May or may not be in the payload.

Dependencies

Identification: None. Independent observation of chat server address from network traffic provides the anchor.

OP_MovementHistory

Purpose

Movement-history-related traffic — exact role uncertain. The handler is unusually large (29 KB of source) and likely accumulates significant logging or analysis logic. Further investigation needed to characterize the opcode's purpose precisely.

Identification evidence

Distinctive content. None.

Frequency. Very low. Sent during session setup or when chat server configuration changes.

Size. Always an array of fixed length structures with a possible trailer. Currently seen as 17N+1 bytes in length.

Direction. C2Z

Context. Fires during the login or zone-in burst, around the same time as other configuration opcodes.

Layout verification evidence

Field layouts are not assumed stable across patches.

Fields to verify:

- Chat server hostname or IP address (ASCII string)
- Port number (numeric)
- Possibly authentication tokens or server identifiers

Operator workflow

Before patch-day operations, characterize this opcode separately. Review Glass/Network/Handlers/HandleMovementHistory.cs to understand what fields are currently extracted and what the handler does with them. Without that characterization, patch-day re-identification of this opcode should be deferred — the handler can be left non-functional through the patch transition without breaking the rest of Glass, since other handlers do not appear to depend on its output.

Known fragilities

Unknown without further investigation.

Dependencies

Unknown without further investigation.

Pre-patch operator checklist

Before a patch lands, ensure:

- Glass database has current state for all characters that will be used in patch-day captures — last-known HP, mana, position, heading, class, level, coin, ability scores
- All characters used in patch-day work have recent zone-in entries (so PlayerProfile / ZoneEntry verification has fresh anchors)
- Prior-patch field definitions are stored in the Inference database — these are the verification reference
- Decompiled signatures for key handler functions from eqgame.exe are archived (Ghidra)
- A capture plan exists for the patch-day session: which zones to enter, which spells to cast, which mobs to kill, which chat channels to use

Patch-day capture plan

A single capture session covering the following behaviors gives material for most opcodes in this document:

1. Login and zone into a populated zone with all six characters — captures PlayerProfile and the ZoneEntry burst
2. Stand still for 30 seconds — establishes ClientUpdate baseline and a sample of NpcMoveUpdate and MobUpdate for mobs in the area
3. Walk a known straight-line path north — provides movement diff data for ClientUpdate coordinate fields

4. Face each cardinal direction in sequence — provides heading anchor for ClientUpdate
5. Activate Track on a ranger or druid — captures Tracking and FormattedMessage with known mob names
6. Target several mobs in sequence — captures Target with known spawn IDs
7. Send chat on multiple channels (/say, /ooc, /shout, /tell) — captures CommonMessage with channel discriminator
8. Cast a known-cost spell on a caster character — captures ManaUpdate with known delta
9. Take damage from a known mob — captures HpUpdate with known delta
10. Kill a mob — captures Death

The capture should be saved with a descriptive filename (date plus patch identifier). The side-car JSON annotation file is created automatically next to it.

After patch-day identification

Once opcodes are identified and field offsets confirmed:

- Write the new opcode values into the patch DLL
- Update field definitions in the Inference database
- Confirm Glass can parse a fresh capture from the new patch end-to-end before resuming normal operation
- Archive the patch-day capture and its annotations as reference for the next patch